University of Hertfordshire UH

School of Physics, Engineering and Computer Science

MSc Data Science Project

7PAM2002

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

County-Level Heart Disease Mortality Classification Using Machine Learning

Student Name and SRN:

Madhagoni Pavan Kalyan - 24082699

Supervisor: Dr. Pushp Raj Tiwari

Date Submitted: 21 April,2026

Word Count: 5,560

Github Link: https://github.com/Pavan240571/heart-disease-mortality.git

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at Assessment Offences and Academic Misconduct and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: MADHAGONI PAVAN KALYAN

Student Name signature: Madhagoni Pavan Kalyan

Student SRN number: 24082699

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Table of Contents

Abstract

This project examines the Heart Disease Mortality Data Among US Adults Aged 35 Years and Older, 2018–2020, to determine whether county-level characteristics can effectively categorize counties by elevated or diminished heart disease mortality rates. The primary objective is to utilize geographic and stratification-related variables from the dataset to develop and evaluate machine learning classification models. The process involves filtering county-level records to remove rows with missing mortality values, creating a binary target based on the median mortality rate, doing exploratory data analysis, and preparing the data for training and testing the model. The results show that both Random Forest and XGBoost are very good at making predictions. Random Forest had an accuracy of 0.899627676078188, while XGBoost had an accuracy of 0.8872168786844554. The results show that machine learning models can find patterns of death in this dataset with a high level of accuracy. They can also help with county-level public health analysis by sorting risks in a way that makes sense and making predictions about the future when people have to make decisions in real life.

Introduction

Heart disease is still a big problem for public health because it kills a lot of people and puts a lot of strain on healthcare systems. Examining patterns of heart disease mortality at the county level can aid in identifying areas with a higher prevalence of heart disease and enhance public health planning and decision-making. This is also a good time to use machine learning on real health data in data science to see how well it can find important patterns.

The Heart Disease Mortality Data Among US Adults Aged 35 Years and Older, 2018–2020 is used in this project to see if county-level traits can be used to group counties into those with high or low heart disease mortality. There are 59,094 rows and 20 columns in the original dataset. The final working dataset has 32,227 entries and 14 columns. It was made by filtering county-level records, taking out rows with missing mortality values, and doing preprocessing. The median mortality rate is used to make a binary target variable. This gives us a class distribution that is almost balanced, with 16,116 low-mortality records and 16,111 high-mortality records.

Aim

To create and compare machine learning classification models that use geographic and stratification-related features at the county level to figure out if heart disease deaths are high or low and see how well they work overall.

Research Question

Can county-level geographic and stratification-related features be used to accurately classify heart disease mortality as high or low?

Objective

- Make the county-level heart disease mortality dataset by keeping the records that matter and getting rid of any missing values.
- Make a binary target variable to divide people with high and low heart disease death rates.
- Look at the data to see how it is set up, how it is spread out, and what the general trends are.
- Make sure the categorical and numerical features are ready to be used in machine learning models.
- Make and compare machine learning models to figure out how many people die from heart disease.
- Use appropriate evaluation metrics to compare the models and identify the best-performing model.

Literature Review

Overview of Heart Disease Mortality Prediction

Heart disease mortality prediction is an important area in data science and public health because it helps identify risk patterns and supports better decision-making. Machine learning is useful in this area because it can detect relationships in health-related data and improve classification performance.

Critical Review

A. Ramathilagam studies the use of machine learning for early coronary heart disease prediction using a Kaggle cardiovascular dataset. The paper includes key steps such as attribute selection, data splitting, normalisation, preprocessing, and class balancing before applying classification models. The main methods include Logistic Regression, Random Forest, Decision Tree, Naïve Bayes, KNN, SVM, and the proposed LRND classifier. This study is relevant because it shows a similar workflow to the present project, especially in preprocessing and model comparison. A clear strength is the use of multiple models. However, the methodological explanation is more general, which makes detailed comparison with this project slightly limited.

S. Prasher uses a Kaggle dataset with 1,025 examples and 14 variables to test how well machine learning algorithms can predict heart disease. The study uses KNN, Multilayer Perceptron, Decision Tree, and REPTree, and it measures performance using accuracy, precision, recall, and F1-score. According to the paper, KNN was the most accurate, with an accuracy rate of 99.7%. This study is helpful because it compares different classification models and uses standard evaluation metrics, which is what this project is doing. One of its best features is that it makes it easy to compare different models. But it is different because it is more about predicting heart disease in general than it is about breaking down deaths by county.

Y. Narayan talks about how AI and ML can help with early prediction and prevention of heart disease. The paper employs a Kaggle heart disease dataset consisting of over 80,000 records and fourteen attributes, subsequently evaluating Medium KNN and SVM Kernel models using metrics such as accuracy, precision, F1-score, specificity, confusion matrices, and ROC analysis. This study is relevant as it illustrates the comparative analysis of machine learning models for cardiovascular prediction using structured health data. One of its best features is that it lets you compare models in the real world. But it covers more ground and isn't as focused on sorting deaths as the current project.

S. Parto's research on forecasting coronary heart disease mortality over a 10 to 15-year span utilizing machine learning is particularly pertinent to this project as it focuses on mortality rather than solely disease prediction. The research utilizes the Sleep Heart Health Study (SHHS) dataset and employs mutual information to select features prior to training the Logistic Regression, Random Forest, KNN, SVM, and Extra Trees Classifier models. The paper says that KNN did the best. One of its best features is that it makes it very clear how to predict death and how important each feature is. But it uses a different population setting and a much smaller sample than this project.

F. Jolha looks into how machine learning can be used to predict how likely it is that ICU patients with heart failure will die in the hospital. The study employs the MIMIC-III dataset comprising 1,177 patients and incorporates preprocessing procedures, including the management of missing values, standardization, and the rectification of class imbalance. I compare a few different models, such as Logistic Regression, SVM, XGBoost, and CatBoost. This paper is important because it focuses on predicting death rather than finding diseases in general. One of its main strengths is that it pays attention to both accuracy and sensitivity. For example, CatBoost has 90% accuracy and 81% sensitivity, but this project is in a different clinical setting.

Comparison with the Present Project

The studies reviewed are relevant to the current project as they illustrate the use of machine learning in forecasting heart disease, evaluating mortality, feature selection, data preprocessing, and model comparison. Logistic Regression, Random Forest, KNN, SVM, XGBoost, and others are some of the most common classification models used in the literature. These studies helped us figure out the best ways to model and test this project. This project is different because it looks at deaths from heart disease at the county level and sorts records into groups based on where they are located and how they are divided up. This makes it clearer that the project is about people and health.

Dataset

Dataset Source

The dataset was obtained from Data.gov, published by the Centers for Disease Control and Prevention. It contains age-standardised heart disease mortality rates for US adults aged 35 and over, reported by state, territory, county, sex, and race/ethnicity during 2018-2020.

Dataset Link: https://catalog.data.gov/dataset/heart-disease-mortality-data-among-us-adults-35-by-state-territory-and-county-2018-2020-3a2b0

Dataset Description

The dataset has heart disease death rates for adults in the United States who are 35 years old or older, adjusted for age. It covers the years 2018 to 2020 and gives mortality rates as 3-year averages, which helps make estimates more stable across areas. The information is sorted by state, territory, and county, and it is also sorted by sex and race/ethnicity. This gives both geographic and demographic information. This means that the dataset not only shows general patterns of death, but it also lets you compare different places and groups of people. It shows the death rate for the whole population instead of just one patient, which makes it useful for public health research on a larger scale. The inclusion of county-level data is very important because it allows for a more localized look at death patterns. This dataset gives a clear and detailed picture of how heart disease deaths differ across the United States. This makes it perfect for this project's classification and pattern analysis.

Exploratory Data Analysis

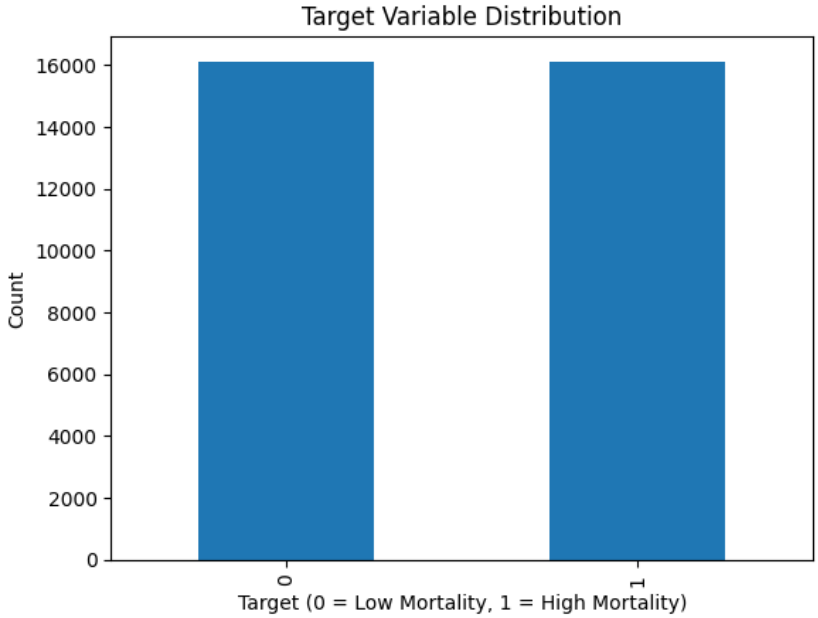

Figure 1: Distribution of the Binary Target Variable

This bar chart shows how the binary target variable, which was made from the median mortality rate, is spread out. Class 0 has 16,116 records of low mortality, and class 1 has 16,111 records of high mortality. At this point, the two classes are almost perfectly balanced, which means that there is no clear class imbalance in the dataset. This is important because it makes the training of the model more fair and the evaluation of its performance more accurate in the classification task.

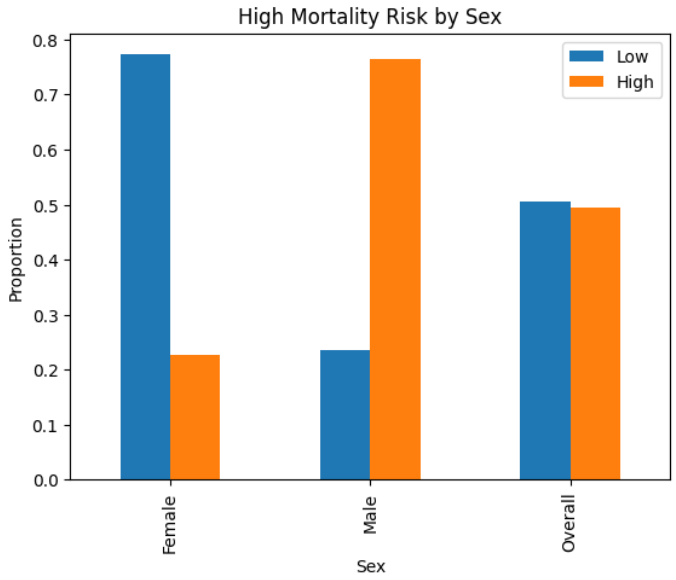

Figure 2: Proportion of High and Low Mortality Risk by Sex

This chart shows how many records of low and high mortality there are in each sex category. There are more female records in the low-mortality group and more male records in the high-mortality group. There are about the same number of people in each class in the Overall category. The pattern indicates that sex could be a valuable attribute for classification, given the significant disparity in mortality risk distribution between female and male groups in the dataset.

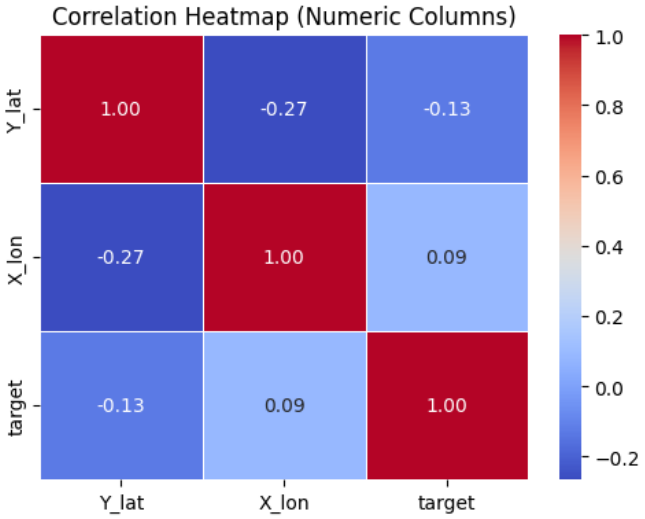

Figure 3: Correlation Heatmap of Numerical Variables

This heatmap shows how the numerical variables Y_lat, X_lon, and target are related to each other in pairs. The diagonal values of 1.00 show that the two things are perfectly correlated with each other. The most significant relationship is a weak negative correlation between Y_lat and X_lon (-0.27). Y_lat (-0.13) and X_lon (0.09) have very weak correlations with the target. This means that these geographic variables don't have a strong linear relationship with the target on their own, so they probably won't be able to explain mortality classification very well.

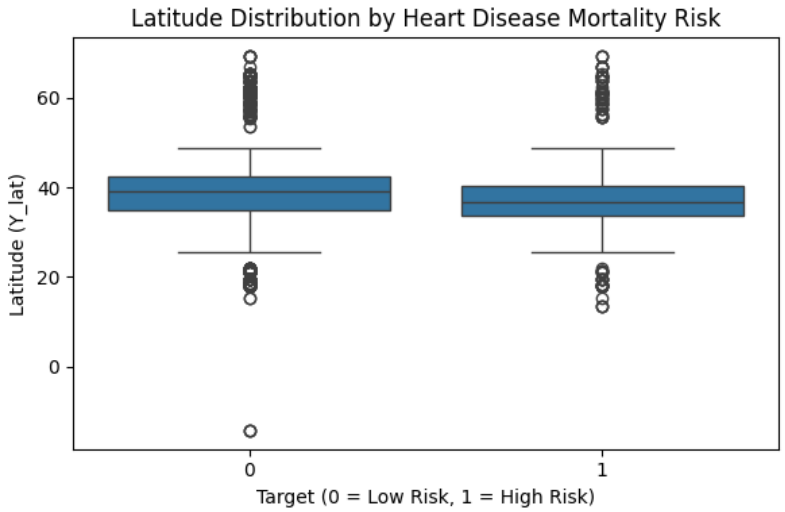

Figure 4: Latitude Distribution by Heart Disease Mortality Risk

This box plot shows how the low-risk (0) and high-risk (1) target groups' latitudes (Y_lat) are spread out. The two groups have very similar interquartile ranges and median values, and their distributions are also very similar. There are outliers in each class at both lower and higher latitudes. This means that latitude alone doesn't always make it clear which records have a low or high risk of death. But it could still be useful when combined with other parts of the classification models.

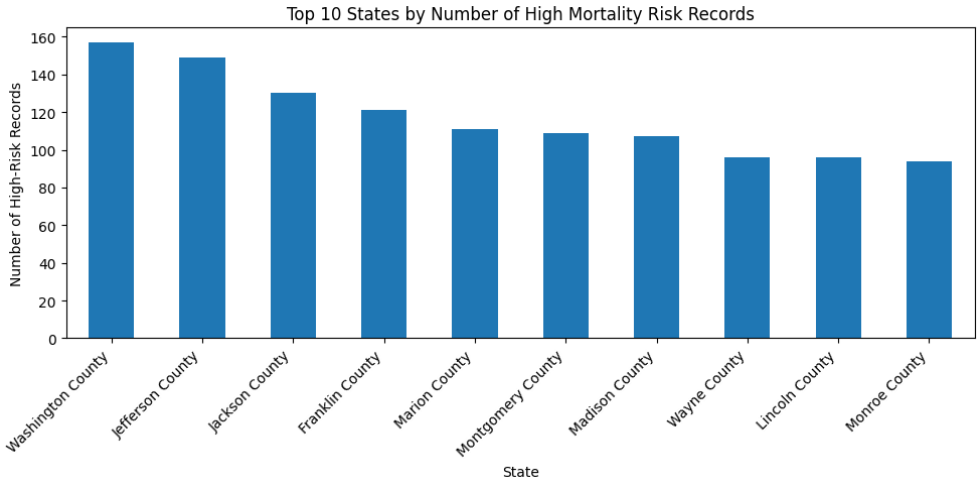

Figure 5: Top 10 Counties by Number of High-Mortality Risk Records

This bar chart shows the ten counties with the most records in the high-mortality risk class. Washington County has the most people, followed by Jefferson County and Jackson County. The other counties also have fairly high frequencies, but the differences are not as big. This plot is helpful for EDA because it shows how the records with a high risk of death are spread out across the counties in the dataset. But it shows the number of records in the dataset, not the real death rate for each county.

Data Preprocessing

Filtering and Cleaning

The first step in preprocessing was to keep only records at the county level, since the project is only interested in county-level heart disease death rates. I then got rid of rows that didn't have a Data_Value because I couldn't use those records to define the outcome variable. I also looked for duplicate rows in the dataset, but none were found. These steps helped make sure that the data used for modeling was complete, relevant, and consistent.

Target Variable Creation

I made a binary target variable from the original mortality values to turn the problem into a classification task. The median mortality rate was used as the cutoff point for this. Records with mortality values higher than the median were given the label "high mortality" (1), and those with values lower than the median were given the label "low mortality" (0). Once the target was made, the original Data_Value column was deleted because it was no longer needed.

Column Preparation and Removal

Some columns were removed during preprocessing because they were metadata, IDs, or not needed for the classification task. These were Data_Value_Unit, Data_Value_Type, Topic, TopicID, DataSource, and LocationID. Some column names were shortened to make it easier to work with the dataset when you analyze and model it. This step helped make the dataset cleaner and more focused for the next step, which is machine learning.

Final Prepared Dataset

After preprocessing, the final dataset contained 32,227 records and 14 columns. It included both categorical and numerical variables, together with the binary target variable used for classification. The target distribution was nearly balanced, with 16,116 records in class 0 and 16,111 records in class 1, which provides a strong basis for fair model training and evaluation. Overall, the preprocessing produced a clean, consistent, and model-ready dataset for the later machine learning analysis.

Personal Data and Privacy

This project employs a secondary dataset that provides age-standardized heart disease mortality rates at the population level. The data are organized by state, territory, county, sex, and race/ethnicity. This means that the dataset does not include any direct personal information, like names, addresses, or phone numbers. The risk

to personal privacy is low because the data are grouped together instead of being at the individual level.

Ethical Issues

Ethical Approval and GDPR

The project does not require gathering data directly from individuals, conducting surveys or interviews, or soliciting anyone to evaluate a system. Because of this, the project didn't need UH ethical approval. Also, the dataset used in this study is not personal-level data, so it doesn't have the same privacy issues as health records that can be linked to a person.

Permission and Responsible Use

The dataset is available to the public and was only used for academic research. Even so, the results need to be understood in a responsible way. The results should not be used to put down certain counties or groups of people, and the conversation should stay focused on patterns at the population level instead of making assumptions about individuals.

Ethical Suitability of the Dataset

In general, this dataset is ethically appropriate for this project because it is aggregated, not identifiable, and useful for public health analysis. This fulfills the handbook's stipulation to contemplate privacy, consent, and the ethical utilization of data, even in the absence of significant ethical concerns.

Methodology

Overview of the Modelling Process

The modeling process began once the dataset was cleaned and the binary target variable was created. The features were then grouped into numerical and categorical variables and prepared for machine learning through preprocessing. Different classification models were trained to predict high and low heart disease mortality. Their performance was then compared using appropriate evaluation metrics. This made it possible to see how well each model classified mortality patterns in the dataset and to identify the model that performed best for the project.

Data Preparation for Modelling

Before the models were built, the input features were separated from the target variable in the dataset to get it ready for the classification task. The median mortality rate was used to make the binary target from the original mortality values. Next, the other variables were split into two groups: categorical and numerical. This way, each type could be handled in the best way. This was necessary because the dataset had both categorical and numerical coordinate values as well as descriptive information. By getting the data ready this way, the dataset was set up correctly and ready for consistent machine learning training and testing.

Feature Encoding and Transformation

I categorized the features into two groups: category and numeric. Next, I used LabelEncoder to make the categorical variables usable in the machine learning models. I saved each encoder for each categorical column so I could use them again later. The numerical variables were then transformed using StandardScaler. The scaler was fitted on the training data and applied to both the training and test sets so that the numerical features were placed on a consistent scale. These encoding and transformation steps were necessary to prepare the mixed feature types in a suitable format for model training and evaluation, and they reflect the exact modeling process used in the notebook.

XGBoost

Extreme Gradient Boosting is a supervised ensemble learning algorithm that uses boosted decision trees. It works by making trees one after the other, with each new tree fixing the mistakes made by the last one. This lets the model find complicated relationships in structured data and works well for classification.

Formula:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i)$$

Where:

- $\hat{y}_i$ = final prediction for the *i*th record
- $f_k(x_i)$ = prediction from each tree
- K = total number of trees

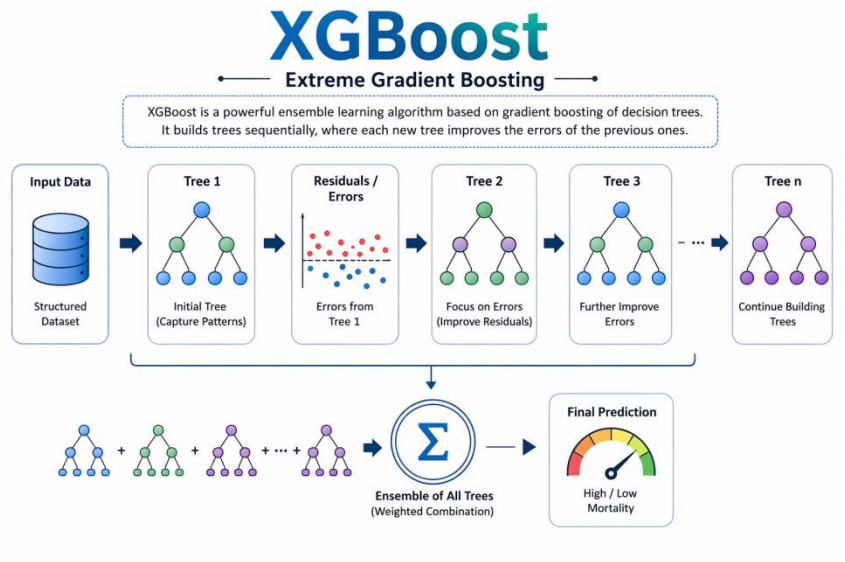

Figure 6: Working Process of the XGBoost Model

It starts with the input data, creates a first decision tree, and then finds the other mistakes. More trees are made one by one to fix those mistakes and make the model better. The final prediction is made by putting all the trees together into one ensemble model. This explains how XGBoost learns from previous mistakes and why it is suitable for structured classification tasks such as predicting high and low heart disease mortality.

Why this Model?

I used XGBoost for this project because the goal was to classify heart disease deaths as either high or low. After preprocessing, the dataset had structured categorical and numerical features that worked well with this model. So, XGBoost was used to see how well a boosting-based classifier could find patterns in deaths and to see how it compared to the other models used in the project.

Base XGBoost parameters:

- n_estimators = 200, max_depth = 5
- learning_rate = 0.1, subsample = 0.8
- colsample_bytree = 0.8
- objective = "binary:logistic"
- eval_metric = "logloss"
- random_state = 42

Best tuned XGBoost parameters:

- n_estimators = 200
- max_depth = 5, learning_rate = 0.2
- subsample = 1.0
- colsample_bytree = 0.8

Random Forest

Random Forest is a supervised ensemble learning algorithm that uses more than one decision tree. A different sample of the training data is used to build each tree, and the final prediction is made by voting. Compared to a single decision tree, this makes things more stable, helps with classification, and cuts down on overfitting.

Formula:

$$\hat{y} = mode\left(T_1(x), T_2(x), \ldots, T_n(x)\right)$$

Where:

- $\hat{y}$ = final predicted class
- $T_1(x), T_2(x), \ldots, T_n(x)$ = predictions from individual trees
- $mode$ = the most frequently predicted class across all trees

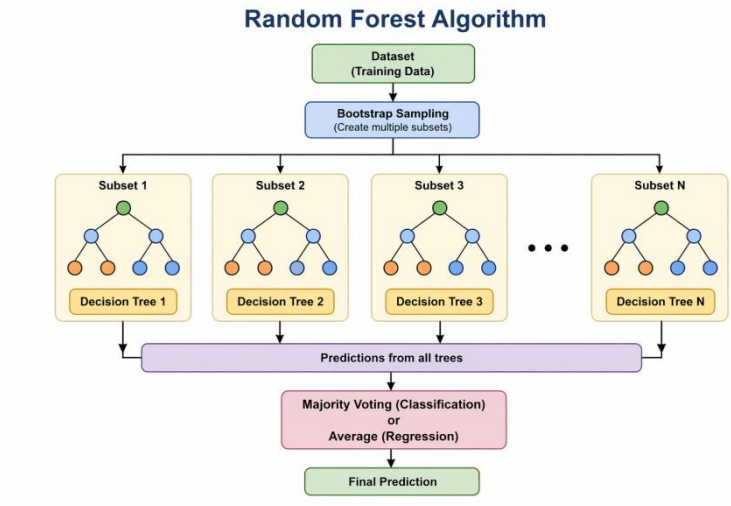

Figure 7: Working Process of the Random Forest Model

This image shows how the Random Forest model works. The first thing that happens is that the input data set is broken up into several bootstrap samples. Thereafter, a new decision tree is made from each sample. Each tree makes its own guess, and the final answer is based on what most people vote for. This process uses the results of many trees instead of just one to make the model's predictions more stable and reliable.

Why this Model?

I used Random Forest for this project because I needed to classify heart disease deaths as either high or low. The dataset had structured categorical and numerical features that were good for this model after preprocessing. So, Random Forest was added to see how well an ensemble tree-based classifier could find patterns of death in the data set and to see how it compared to the other models used in the project.

Base Random Forest parameters

- n_estimators = 200
- max_depth = None
- random_state = 42
- n_jobs = -1

Best tuned Random Forest parameters

- n_estimators = 200
- min_samples_split = 2
- min_samples_leaf = 1
- max_depth = None

K-Nearest Neighbors

K-Nearest Neighbors is a supervised machine learning algorithm that is used to sort things into groups. It doesn't make a complicated model while it's learning. It does not do this. Instead, it sorts a new record by measuring how far it is from other records and giving it the class that is most common among its k nearest neighbors.

Formula:

$$d(x, x_i) = \sqrt{\Sigma_{(j=1)}^{n}}\left(x_j - x_{(ij)}\right)^2$$

Where:

- $d(x, x_i)$ = distance between the new record and a training record
- x_j = value of the new record for feature *j*

- $x_{(ij)}$ = value of the *i*th training record for feature *j*
- n = total number of features

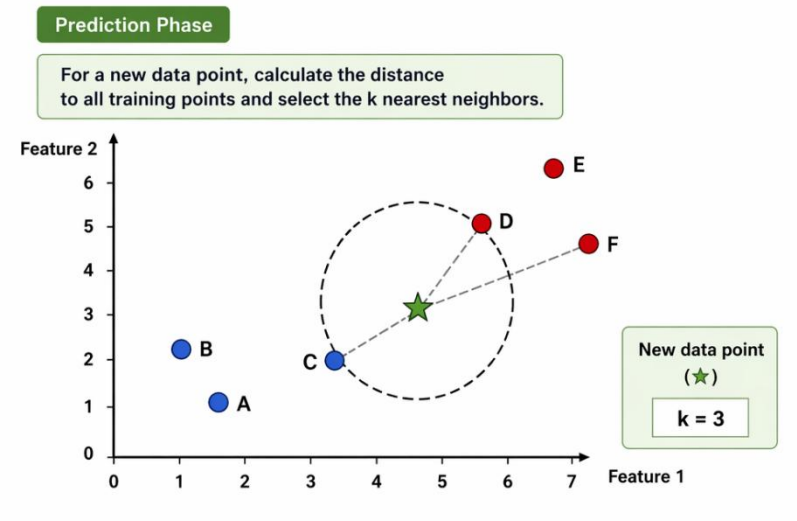

Figure 8: Prediction Stage of the K-Nearest Neighbors (KNN) Model

This diagram shows the prediction stage of KNN. For a new data point, the model calculates its distance from all training points and selects the k nearest neighbours, where k = 3 in this example. The classes of these nearest neighbours are then compared, and the final class is assigned by majority vote. This shows how KNN classifies a new record based on similarity to nearby points in the feature space.

Why this Model?

I used KNN for this project because I needed to sort heart disease deaths into high and low groups. The dataset was ready for distance-based classification after it had been preprocessed. KNN was added to see how well a simple instance-based model could find death patterns and to make it easier to compare with the other models used in the project.

Base KNN parameters

- n_neighbors = 5
- weights = "distance"

Best tuned KNN parameters

- n_neighbors = 11
- weights = "distance"
- metric = "manhattan"

Model Evaluation Metrics

I used accuracy, precision, recall, and F1-score to judge the models. Accuracy measured how many of the predictions were right. Precision told us how many of the predicted positive cases were actually correct, and recall told us how well the model found the true positive class. The F1-score was a single balanced measure that combined precision and recall. These metrics were appropriate because the project involved a binary classification task, and the handbook says that the right metrics must be used to clearly and accurately evaluate results.

Hyperparameter Tuning

I fine-tuned the hyperparameters to enhance the model and ensure a fair comparison of the project's classification models.I tried different settings for XGBoost, Random Forest, and K-Nearest Neighbors (KNN) to find the best combination that worked best for us. This step was important because the model's behavior can change depending on the values of the parameters you choose. After tuning, the models were tested with the same metrics to make sure that their performance could be compared fairly. This comparison was part of the modeling process and helped find the best way to tell if heart disease deaths are high or low.

Formula:

$$\theta^* = \arg\max_{\theta \in \Theta} S\,c\,ore(M_\theta)$$

Where:

- θ^* = best set of hyperparameters
- Θ = all tested parameter combinations
- M_θ = model trained with a given parameter set
- $Score$ = evaluation metric used to choose the best model

Results

Base Model Results

A base model is the first version of a machine learning model that has been trained with its default settings, before any hyperparameter tuning has been done. It is used to set a baseline for performance so that the effects of later tuning can be clearly seen and compared across models.

Model	Accuracy	Precision (Class 0)	Precision (Class 1)	Recall (Class 0)	Recall (Class 1)	F1-Score (Class 0)	F1-Score (Class 1)
XGBoost	0.8872168786844554	0.89	0.88	0.88	0.89	0.89	0.89
Random Forest	0.899627607607818 8	0.90	0.90	0.91	0.89	0.90	0.90
KNN	0.6552901023890785	0.66	0.65	0.64	0.68	0.65	0.66

Table 1: Performance Comparison of the Base Models

Before tuning the hyperparameters, this table shows how well the three base models did. The Random Forest model had the best base accuracy (0.899627676078188), followed by the XGBoost model (0.8872168786844554). The KNN model had the worst accuracy (0.6552901023890785). The F1-score, recall, and precision values also show that Random Forest and XGBoost were more reliable than KNN in both classes. In general, the table is a good place to start when comparing the models before tuning was done.

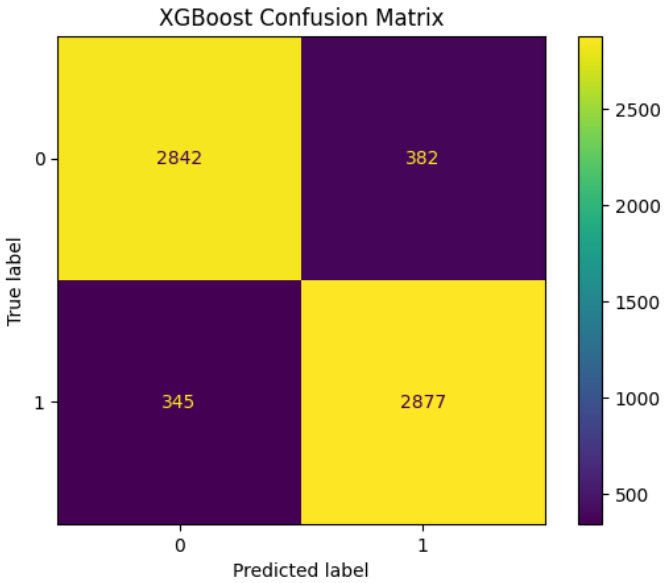

Figure 9: XGBoost Confusion Matrix

The XGBoost model made these true and predicted classes, which you can see in this confusion matrix. It correctly sorted 2842 records with low mortality and 2877 records with high mortality. It incorrectly labeled 382 records with low mortality as high mortality and 345 records with high mortality as low mortality. Overall, most of the records were put in the right category, which shows that XGBoost did well in both classes in this dataset.

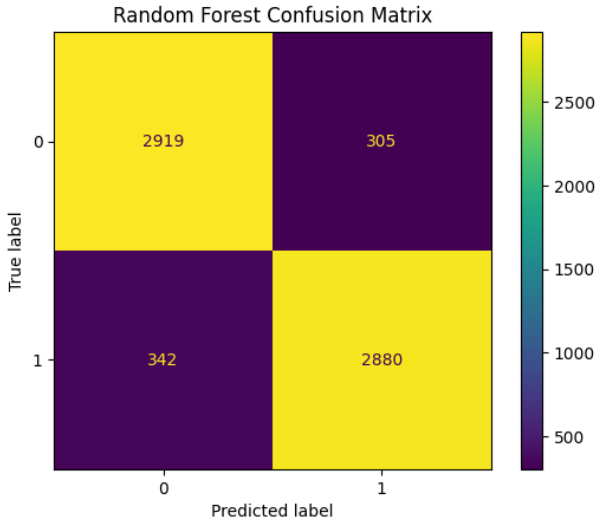

Figure 10: Random Forest Confusion Matrix

The Random Forest model used this confusion matrix to sort the two groups of deaths. It correctly found 2919 records with low mortality and 2880 records with high mortality. It also made some mistakes, putting 305 low-mortality records in the high-mortality category and 342 high-mortality records in the low-mortality category. In general, the model did a good job of separating low- and high-mortality patterns in the dataset because most records were put in the right class.

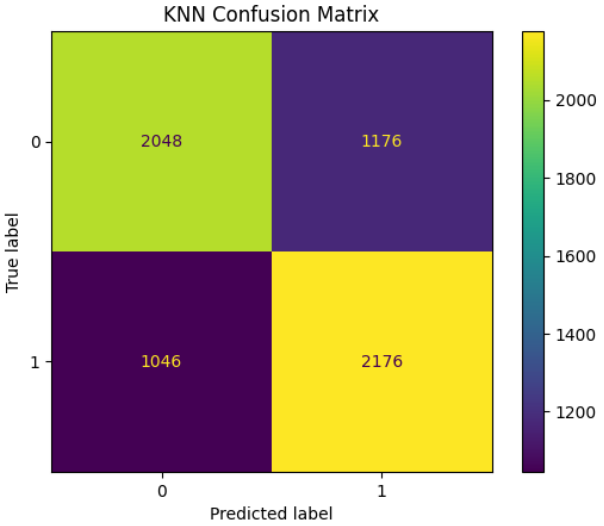

Figure 11: KNN Confusion Matrix

The KNN model made these true and predicted classes, which you can see in this confusion matrix. It correctly put 2048 records with low mortality and 2176 records with high mortality into the right groups. It incorrectly labeled 1176 low-mortality records as high mortality and 1046 high-mortality records as low mortality. KNN made more mistakes than the other models, which means it did a worse job of classifying both classes in this dataset.

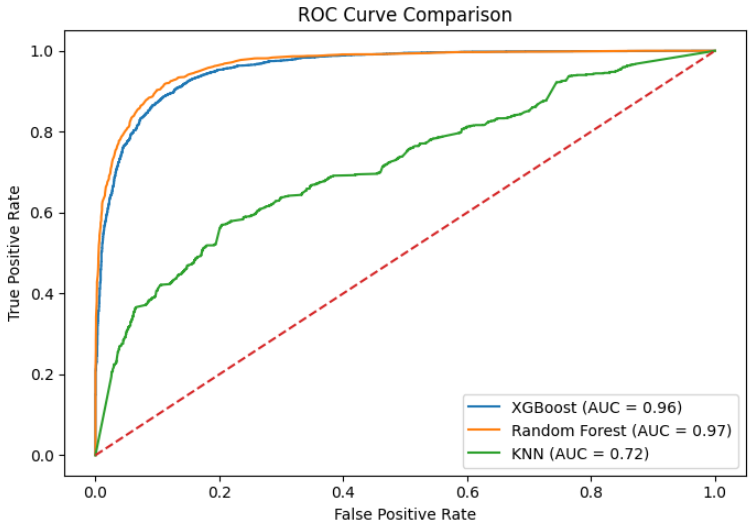

Figure 12: ROC Curve Comparison of the Models

This ROC curve shows how well the three models separated the two classes at different levels of classification. The red dashed diagonal line shows how well random classification works. The AUC for Random Forest was the highest at 0.97, followed closely by XGBoost at 0.96. KNN, on the other hand, did not do as well at 0.72. The curves for Random Forest and XGBoost stay much closer to the top-left corner, which means they can tell the difference between records with low and high mortality rates better than KNN.

Hyperparameter tuning

Hyperparameter tuning was used to adjust the model settings so that each model could work better with the dataset. In the Results section, it is included to show the performance after tuning and to compare it with the base models. This helps show clearly whether tuning improved the models and led to better classification results.

Model	Best Tuned Parameters	Accuracy	ROC-AUC
XGBoost	subsample=1.0, n_estimators=200, max_depth=5, learning_rate=0.2, colsample_bytree=0.8	0.89	0.9616357879220557
Random Forest	n_estimators=200, min_samples_split=2, min_samples_leaf=1, max_depth=None	0.90	0.9655757736436688
KNN	weights=distance, n_neighbors=11, metric=manhattan	0.69	0.7594930286969392

Table 2: Performance Comparison of the Tuned Models

This table presents the performance of the tuned classifiers used in the project after hyperparameter tuning. The classifiers included are XGBoost, Random Forest, and KNN. For each model, the table reports the best tuned parameters, accuracy, and

ROC-AUC, allowing their final performance to be compared clearly. The results show that Random Forest achieved the strongest overall performance, followed by XGBoost, while KNN produced the lowest performance among the three tuned models.

Comparison of Base and Tuned Models

Model	Base Accuracy	Tuned Accuracy
XGBoost	0.8872168786844554	0.89
Random Forest	0.899627676078188	0.90
KNN	0.6552901023890785	0.69

Table 3: Comparison of Base and Tuned Model Accuracy

This table shows how the accuracy of the three models changed before and after the hyperparameters were tuned. Random Forest was the most accurate in both stages, and XGBoost was very close behind it. KNN was the least accurate overall, but its tuned version was better than the base model. The table makes it easy to compare because it only shows the base and tuned accuracy values.

Analysis and Discussion

Overall Model Performance

The results show that the three models worked very differently. The base accuracy of Random Forest was the best (0.899627676078188) and the tuned accuracy was the best (0.90). XGBoost was next, with a base accuracy of 0.8872168786844554 and a tuned accuracy of 0.89. KNN got the worst results, going from 0.6552901023890785 to 0.69 after tuning. This shows that the tree-based ensemble models did a much better job of classifying heart disease deaths in this dataset than KNN did.

Justification of the Best-Performing Model

Random Forest was the strongest model in this project, and this is clear from both its accuracy and confusion matrix. It correctly classified 2919 low-mortality records and 2880 high-mortality records, while making relatively few mistakes, with 305 and 342 misclassifications. This shows that the model performed well across both classes. A likely reason is that Random Forest combines many decision trees, which helps it learn more complex patterns in the data while also reducing overfitting. This made it a good fit for a structured dataset with geographic and stratification-related features.

Comparison of the Models

The comparison shows that Random Forest and XGBoost did about the same, but KNN was still much weaker. Random Forest got an AUC of 0.97 on the ROC curve, XGBoost got 0.96, and KNN got 0.72. This means that the two ensemble models were much better at separating classes. The difference between KNN and the other two models shows that tree-based ensemble methods worked better with the dataset than a distance-based classifier.

Effect of Hyperparameter Tuning

Hyperparameter tuning didn't change the order of the models, but it did help prove which one was the best. Random Forest stayed in first place, XGBoost stayed in second place, and KNN stayed in third place. KNN showed the most improvement, going from 0.6552901023890785 to 0.69. Even after tuning, though, KNN still did much worse than Random Forest (0.90) and XGBoost (0.89). This means that for this classification task, model suitability was more important than just tuning.

Relation to the Research Question and Objectives

The research question inquired if county-level geographic and stratification-related characteristics could effectively categorize heart disease mortality as elevated or diminished. The results demonstrate that this objective was successfully attained. The high scores of Random Forest (0.90) and XGBoost (0.89) show that the features that were available had enough information to effectively separate the two groups of people who died. This means that the project goal was met and that the goals were met through preprocessing, model development, evaluation, and comparison.

Limitations, Impact, and Importance of the Results

Notwithstanding the strong results, they must be framed within the study's constraints. The target variable was defined by dividing the original mortality values into two groups based on the median, so streamlining the classification procedure and reducing the granularity of the original continuous data. The resultant dataset had 32,227 entries and 14 columns, sourced from county-level aggregated data rather than individual-level observations. Under these conditions, the results remain substantial. Random Forest had superior performance, achieving a base accuracy of 0.899627676078188 and a tuned accuracy of 0.90, while XGBoost also displayed commendable performance, with a base accuracy of 0.8872168786844554 and a tuned accuracy of 0.89. KNN demonstrated subpar performance, improving from 0.6552901023890785 to 0.69 following tuning. The results demonstrate that the selected county-level attributes sufficiently distinguished between high- and low-mortality groups, hence validating the effectiveness of machine learning for county-level public health assessments and wider population-level decision-making.

Comparison with the Literature

The results of this project are similar to those of other studies that have been looked at. In this project, Random Forest and XGBoost worked the best, while KNN worked much worse. This is in line with research by A. Ramathilagam, S. Prasher, and F. Jolha that found that stronger tree-based or ensemble models also worked well on heart disease data. In S. Parto, it's easy to see that KNN was the best model. This shows that the performance of a model can change based on the dataset, the features used, and the goal of the prediction. Overall, the project's results fit well with what other researchers have found. They also show why it's important to compare different models.

Conclusion

This project investigated whether county-level geographic and stratification-related features could be used to classify heart disease mortality as **high** or **low**. The findings show that this was achieved successfully through machine learning classification. After data cleaning, exploratory analysis, preprocessing, model development, and evaluation, the project demonstrated that the selected features contained useful information for distinguishing between the two mortality groups.

The Random Forest model did the best overall out of the three that were used. It got a base accuracy of 0.899627676078188 and a tuned accuracy of 0.90. XGBoost also did well, with a base accuracy of 0.8872168786844554 and a tuned accuracy of 0.89. KNN, on the other hand, had the worst results, but they got better after tuning from 0.6552901023890785 to 0.69. The ROC curve and confusion matrices backed up these results, showing that Random Forest and XGBoost did a much better job of separating the two classes than KNN did.

These results show that the tree-based ensemble models worked better for this dataset than the distance-based classifier. This is likely because the dataset includes structured geographic and stratification-related variables, and the ensemble models were better able to pick up the more complex patterns in them. This means the project successfully answered the research question and achieved its main aim of building and comparing machine learning models for heart disease mortality classification.

These findings demonstrate that machine learning can assist county-level public health analysis by elucidating mortality patterns among various geographic and demographic groups. The dataset is based on data from all the counties in the US, so the results should be seen as patterns in the population as a whole, not as predictions about specific people. The project could be better in the future by trying out more models, using more detailed features, or predicting death as a continuous value instead of breaking it up into two groups.

References

Jolha, F. and Jolha, N. (2024) 'Predictive Modeling of In-Hospital Mortality in ICU Heart Failure Patients Using Machine Learning Techniques', in 2024 7th International Conference on Algorithms, Computing and Artificial Intelligence (ACAI). Guangzhou, China, pp. 01–07. doi: 10.1109/ACAI63924.2024.10899560.

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10899560&isnumber=10899447

Prasher, S., Nelson, L. and Hariharan, S. (2023) 'Evaluation of Machine Learning Algorithms for Heart Disease Prediction in Healthcare', in 2023 International Conference on Innovations in Engineering and Technology (ICIET). Muvattupuzha, India, pp. 1–4. doi: 10.1109/ICIET57285.2023.10220917.

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10220917&isnumber=10220577

Parto, S., Safavi, A. A., Naghsh, S., Keikha, M. and Sharafkhaneh, A. (2025) 'Predicting Coronary Heart Disease Mortality Using Machine Learning Algorithms', in 2025 Conference on Artificial Intelligence x Multimedia (AIxMM). Laguna Hills, CA, USA, pp. 35–38. doi: 10.1109/AIxMM62960.2025.00012.

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=11005061&isnumber=11004851

Narayan, Y., Kumar, N., Kumar, V., Manya and Karan, D. (2025) 'Heart Disease Identification Using Machine Learning Algorithm', in 2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI). Gwalior, India, pp. 1–4. doi: 10.1109/IATMSI64286.2025.10985429.

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10985429&isnumber=10984438

Ramathilagam, A., Pradeepha, S., Ram, M. P., Lakshmi, B. S., Kumar, S. A. and Neels Ponkumar, D. D. (2024) 'A Method for Predicting Coronary Heart Disease Using Machine Learning Approaches', in 2024 2nd International Conference on Recent Trends in Microelectronics, Automation, Computing and Communications Systems (ICMACC). Hyderabad, India, pp. 231–235. doi: 10.1109/ICMACC62921.2024.10894635.

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10894635&isnumber=10893896

Appendix:

```
import pandas as pd

# Load dataset
df = pd.read_csv(

"Heart_Disease_Mortality_Data_Among_US_Adults__35___by_State_Territory_and
_County___2018-2020.csv"
)

# Check shape and column names
print(df.shape)
print(df.columns)

# Filter only county-level records
df = df[df["GeographicLevel"] == "County"]

# Drop rows with missing mortality rate
df = df[df["Data_Value"].notna()]

# Calculate median mortality rate
median_mortality = df["Data_Value"].median()

# Create binary target column
df["target"] = (df["Data_Value"] > median_mortality).astype(int)

# Remove continuous target column
df = df.drop(columns=["Data_Value"])
```

```
# Columns to drop (metadata / identifiers)
drop_cols = [
    "Data_Value_Unit",
    "Data_Value_Type",
    "Low_Confidence_Limit",
    "High_Confidence_Limit",
    "Topic",
    "TopicID",
    "DataSource",
    "LocationID"
]

df = df.drop(columns=drop_cols, errors="ignore")

# Rename columns for cleaner coding
df = df.rename(columns={
    "LocationAbbr": "state_abbr",
    "LocationDesc": "state_name",
    "GeographicLevel": "geo_level",
    "County": "county",
    "Race": "race",
    "Sex": "sex",
    "AgeGroup": "age_group"
})

# Check duplicate rows
print("Duplicate rows:", df.duplicated().sum())

# Remove duplicates
```

```
df = df.drop_duplicates()

# Final overview
df.info()
df.head()

# Check class distribution
df["target"].value_counts()

# Plot target distribution
import matplotlib.pyplot as plt

plt.figure()
df["target"].value_counts().plot(kind="bar")
plt.xlabel("Target (0 = Low Mortality, 1 = High Mortality)")
plt.ylabel("Count")
plt.title("Target Variable Distribution")
plt.show()

# Filter rows where stratification is Sex
df_sex = df[df["StratificationCategory1"] == "Sex"]

# Check mortality risk by sex
pd.crosstab(df_sex["Stratification1"], df_sex["target"], normalize="index")


# Plot
import matplotlib.pyplot as plt
```

```
pd.crosstab(df_sex["Stratification1"], df_sex["target"],
normalize="index").plot(kind="bar")

plt.xlabel("Sex")

plt.ylabel("Proportion")

plt.title("High Mortality Risk by Sex")

plt.legend(["Low", "High"])

plt.show()


# Check the exact category names available

print(df["StratificationCategory1"].value_counts())

print(df["StratificationCategory1"].unique())

# ------------------------------

# Heatmap using numerical columns (excluding Year)

# ------------------------------


import seaborn as sns

import matplotlib.pyplot as plt


# Select numeric columns

numeric_df = df.select_dtypes(include=["int64", "float64"])


# Drop Year because it is a time index, not a true feature

numeric_df = numeric_df.drop(columns=["Year"])


# Compute correlation matrix

corr_matrix = numeric_df.corr()


# Plot heatmap

plt.figure(figsize=(5,4))
```

```
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)

plt.title("Correlation Heatmap (Numeric Columns)")
plt.tight_layout()
plt.show()

# ------------------------------
# Box plot: Latitude vs Mortality Risk
# ------------------------------

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))

# Box plot comparing latitude distribution by target class
sns.boxplot(
    x="target",
    y="Y_lat",
    data=df
)

plt.xlabel("Target (0 = Low Risk, 1 = High Risk)")
```

```
plt.ylabel("Latitude (Y_lat)")
plt.title("Latitude Distribution by Heart Disease Mortality Risk")
plt.tight_layout()
plt.show()

# ------------------------------
# Plot 1: Count of high-risk records by state
# ------------------------------

import matplotlib.pyplot as plt

# Count high-risk records per state
state_high_risk_counts = (
    df[df["target"] == 1]
    .groupby("state_name")
    .size()
    .sort_values(ascending=False)
)

# Plot top 10 states
plt.figure(figsize=(10,5))
state_high_risk_counts.head(10).plot(kind="bar")

plt.xlabel("State")
plt.ylabel("Number of High-Risk Records")
plt.title("Top 10 States by Number of High Mortality Risk Records")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
# Separate features and target
X = df.drop(columns=["target"])
y = df["target"]

# Categorical columns (object type)
categorical_cols = X.select_dtypes(include="object").columns

# Numerical columns (exclude target)
numerical_cols = X.select_dtypes(include=["int64", "float64"]).columns
print("Categorical columns:", categorical_cols.tolist())
print("Numerical columns:", numerical_cols.tolist())

# Dictionary to store encoders
label_encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    label_encoders[col] = le

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y  # preserves class balance
```

```
)

# Initialize scaler
scaler = StandardScaler()

# Fit on training data, transform both
X_train[numerical_cols] = scaler.fit_transform(X_train[numerical_cols])
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])

print("Training set shape:", X_train.shape)
print("Test set shape:", X_test.shape)

# ------------------------------
# Random Forest Model
# ------------------------------

from sklearn.ensemble import RandomForestClassifier

# Initialize model
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    random_state=42,
    n_jobs=-1
)

# Train model
rf_model.fit(X_train, y_train)
```

```
# Predictions
rf_pred = rf_model.predict(X_test)

# Evaluation
print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))
print("\nRandom Forest Classification Report:\n")
print(classification_report(y_test, rf_pred))

# ------------------------------
# KNN Model
# ------------------------------

from sklearn.neighbors import KNeighborsClassifier

# Initialize model
knn_model = KNeighborsClassifier(
    n_neighbors=5,
    weights="distance"   # improves performance
)

# Train model
knn_model.fit(X_train, y_train)

# Predictions
knn_pred = knn_model.predict(X_test)

# Evaluation
print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
print("\nKNN Classification Report:\n")
```

```
print(classification_report(y_test, knn_pred))

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm_xgb = confusion_matrix(y_test, xgb_pred)

disp_xgb = ConfusionMatrixDisplay(confusion_matrix=cm_xgb)
disp_xgb.plot()
plt.title("XGBoost Confusion Matrix")
plt.show()


cm_rf = confusion_matrix(y_test, rf_pred)

disp_rf = ConfusionMatrixDisplay(confusion_matrix=cm_rf)
disp_rf.plot()
plt.title("Random Forest Confusion Matrix")
plt.show()

cm_knn = confusion_matrix(y_test, knn_pred)

disp_knn = ConfusionMatrixDisplay(confusion_matrix=cm_knn)
disp_knn.plot()
plt.title("KNN Confusion Matrix")
plt.show()

from sklearn.metrics import roc_curve, auc
# Get prediction probabilities
```

```
xgb_probs = xgb_model.predict_proba(X_test)[:, 1]
rf_probs = rf_model.predict_proba(X_test)[:, 1]
knn_probs = knn_model.predict_proba(X_test)[:, 1]

# Compute ROC curves
fpr_xgb, tpr_xgb, _ = roc_curve(y_test, xgb_probs)
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_probs)
fpr_knn, tpr_knn, _ = roc_curve(y_test, knn_probs)

# Compute AUC scores
auc_xgb = auc(fpr_xgb, tpr_xgb)
auc_rf = auc(fpr_rf, tpr_rf)
auc_knn = auc(fpr_knn, tpr_knn)

# Plot ROC curves
plt.figure(figsize=(7,5))
plt.plot(fpr_xgb, tpr_xgb, label=f"XGBoost (AUC = {auc_xgb:.2f})")
plt.plot(fpr_rf, tpr_rf, label=f"Random Forest (AUC = {auc_rf:.2f})")
plt.plot(fpr_knn, tpr_knn, label=f"KNN (AUC = {auc_knn:.2f})")
plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.tight_layout()
plt.show()

from sklearn.metrics import accuracy_score
```

```
# Accuracy values
accuracy_scores = {
    "XGBoost": accuracy_score(y_test, xgb_pred),
    "Random Forest": accuracy_score(y_test, rf_pred),
    "KNN": accuracy_score(y_test, knn_pred)
}

# Plot accuracy comparison
plt.figure(figsize=(6,4))
plt.bar(accuracy_scores.keys(), accuracy_scores.values())

plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.title("Model Accuracy Comparison")
plt.tight_layout()
plt.show()

# Import required tools
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.metrics import roc_auc_score, classification_report

# Stratified cross-validation (keeps class balance in folds)
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

# ------------------------------
# XGBoost Hyperparameter Tuning
# ------------------------------
```

```
from xgboost import XGBClassifier

# Initialize base XGBoost model
xgb_model = XGBClassifier(
    objective="binary:logistic",
    eval_metric="logloss",
    random_state=42,
    n_jobs=-1
)

# Define parameter search space (kept small for speed)
xgb_param_dist = {
    "n_estimators": [100, 200, 300],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1, 0.2],
    "subsample": [0.8, 1.0],
    "colsample_bytree": [0.8, 1.0]
}

# RandomizedSearchCV for faster tuning
xgb_search = RandomizedSearchCV(
    estimator=xgb_model,
    param_distributions=xgb_param_dist,
    n_iter=10,              # small number of trials (FAST)
    scoring="roc_auc",
    cv=cv,
    random_state=42,
    n_jobs=-1
)
```

```
# Fit tuning on training data only
xgb_search.fit(X_train, y_train)

# Get best tuned model
best_xgb = xgb_search.best_estimator_

print("Best XGBoost Parameters:", xgb_search.best_params_)

# Evaluate on test data
xgb_probs = best_xgb.predict_proba(X_test)[:, 1]
xgb_pred = best_xgb.predict(X_test)

print("XGBoost ROC-AUC:", roc_auc_score(y_test, xgb_probs))
print(classification_report(y_test, xgb_pred))



# ------------------------------
# Random Forest Hyperparameter Tuning
# ------------------------------

from sklearn.ensemble import RandomForestClassifier

# Initialize base Random Forest model
rf_model = RandomForestClassifier(
    random_state=42,
    n_jobs=-1
)
```

```
# Parameter search space (reduced for speed)
rf_param_dist = {
    "n_estimators": [200, 300, 500],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 5]
}

# RandomizedSearchCV
rf_search = RandomizedSearchCV(
    estimator=rf_model,
    param_distributions=rf_param_dist,
    n_iter=10,              # FAST
    scoring="roc_auc",
    cv=cv,
    random_state=42,
    n_jobs=-1
)

# Fit tuning
rf_search.fit(X_train, y_train)

# Best tuned model
best_rf = rf_search.best_estimator_

print("Best Random Forest Parameters:", rf_search.best_params_)

# Evaluate on test data
rf_probs = best_rf.predict_proba(X_test)[:, 1]
```

```
rf_pred = best_rf.predict(X_test)

print("Random Forest ROC-AUC:", roc_auc_score(y_test, rf_probs))
print(classification_report(y_test, rf_pred))

# ------------------------------
# KNN Hyperparameter Tuning
# ------------------------------

from sklearn.neighbors import KNeighborsClassifier

# Initialize base KNN model
knn_model = KNeighborsClassifier()

# Parameter search space
knn_param_dist = {
    "n_neighbors": [3, 5, 7, 9, 11, 15],
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "manhattan"]
}

# RandomizedSearchCV
knn_search = RandomizedSearchCV(
    estimator=knn_model,
    param_distributions=knn_param_dist,
    n_iter=10,              # FAST
    scoring="roc_auc",
    cv=cv,
    random_state=42,
```

```
    n_jobs=-1
)

# Fit tuning
knn_search.fit(X_train, y_train)

# Best tuned model
best_knn = knn_search.best_estimator_

print("Best KNN Parameters:", knn_search.best_params_)

# Evaluate on test data
knn_probs = best_knn.predict_proba(X_test)[:, 1]
knn_pred = best_knn.predict(X_test)

print("KNN ROC-AUC:", roc_auc_score(y_test, knn_probs))
print(classification_report(y_test, knn_pred))
```